

# Anwendung von Reinforcement Learning zur Entwicklung authentischer Spielwelten mithilfe bedürfnisbasierter Agenten

Masterthesis von Tobias Jansing

# Agenda

1. Überblick
2. Grundlagenthemen
3. Reinforcement Learning
4. Implementierung des Dorfsystems
5. Ergebnisse
6. Demonstration
7. Fazit



# 1. Überblick

# Idee

“Ein alternativer Ansatz zur Entwicklung von Non-Player-Characters anhand einer Dorfsimulation mit Unity”



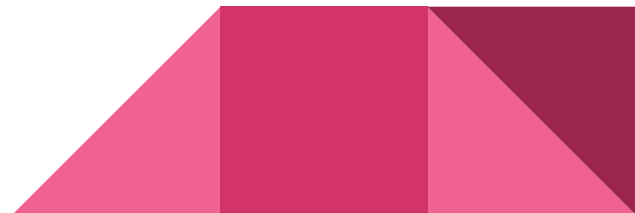
# Idee

- Neuer Ansatz für glaubwürdigere Darstellung von Hintergrundcharakteren
- Entwicklung eines Dorfsystems als Umgebung für NPCs
- Bedürfnisorientierte Agenten, die auf Reinforcement Learning basieren



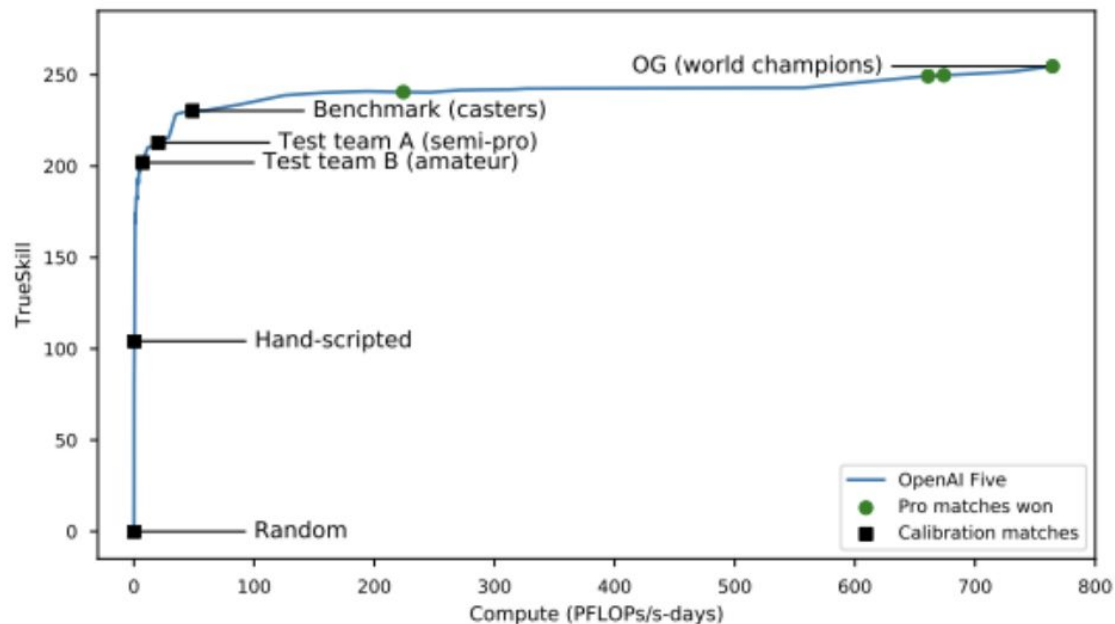
# Motivation

- Virtuelle Welten werden immer realistischer
- NPCs wirken trotzdem oft “dumm”, haben keinen Tagesablauf
- Hoher Aufwand für Scripting von NPCs





# Motivation





# Ziele

- Implementierung eines Dorfsystems mit Agenten
- NPCs haben Bedürfnisse, die ihre Entscheidungen beeinflussen
- Verhalten der NPC soll authentischer und weniger statisch wirken
- Reduzierung des nötigen Aufwands
- Entstehung eines Tagesablaufs

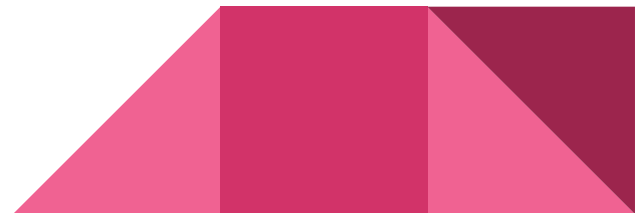




## 2. Grundlagenthemen

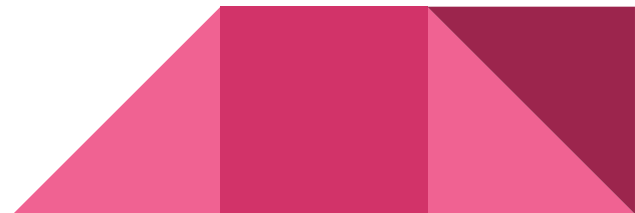
# Intelligente Agenten in Spielen

- Autonom handelnde Entitäten wie NPCs
- Simulierte Intelligenz, Agent soll intelligent wirkende Entscheidungen treffen
- Unterschiedliche Verfahren zur Implementierung möglich
  - Finite State Machines
  - Decision Trees
  - Behavior Trees
  - Goal Oriented Action Planning
  - Utility Based Agents



# Weitere Grundlagenthemen

- Reinforcement Learning
  - Einführung, Abgrenzung
  - Mathematische Grundlagen
  - Beschreibung verschiedener Verfahren
- Machine Learning in Videospielen
  - Auflistung und Beschreibung von Beispielen
  - Aktuelle Entwicklungen

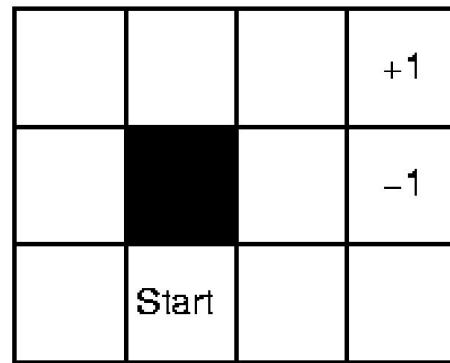




# 3. Reinforcement Learning

# Reinforcement Learning

- Lösung von Markov-Entscheidungsproblemen durch Lernvorgang
- Aktionen ändern Zustand und resultiert in Belohnung / Bestrafung
- Ziel: Maximierung von Belohnungsfunktion
- Lernvorgang durch “Ausprobieren”
  - Anpassung von Parametern (z.B. in neuronalem Netz)
  - Anpassung basierend auf Erfahrungen
- Trainingsvorgang erzeugt Modell bzw. *Policy*

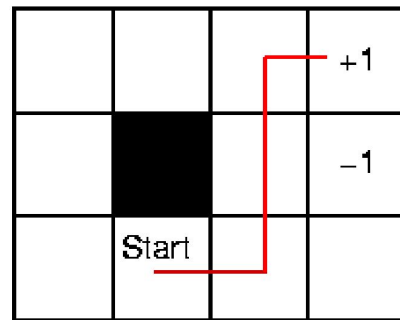
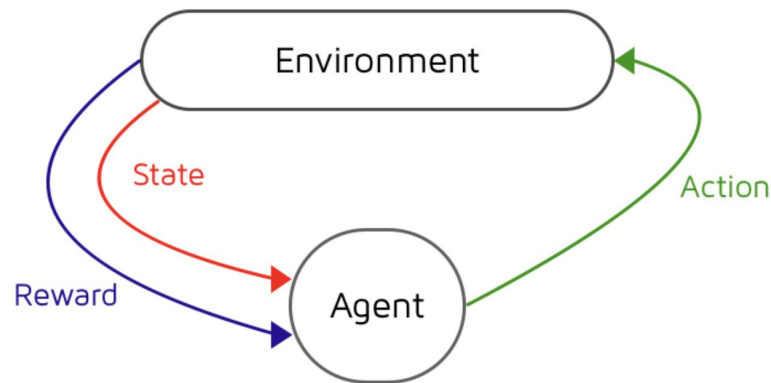


|  |       |  |    |
|--|-------|--|----|
|  |       |  | +1 |
|  |       |  | -1 |
|  | Start |  |    |

# Reinforcement Learning

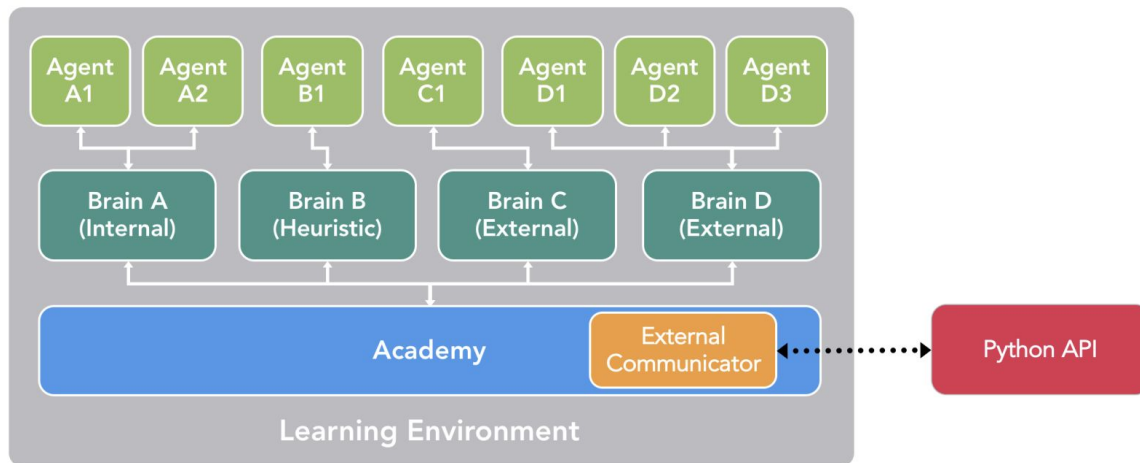
- Früher eher modellbasiert, heute eher modellfrei
- Modell ist Wahrscheinlichkeitsverteilung von Aktionen über Zuständen
- Kombination von Zustandsübergängen heißt *Trajectory*  $\tau$
- Gesamtbelohnung einer Policy:

$$J(\theta) = \int p(\tau|\theta) R(\tau) d\tau$$



# ML-Agents Toolkit

- Bibliothek für Machine Learning in Unity
- Fokus auf Reinforcement Learning
- Beta-Phase, aber bereits gut dokumentiert



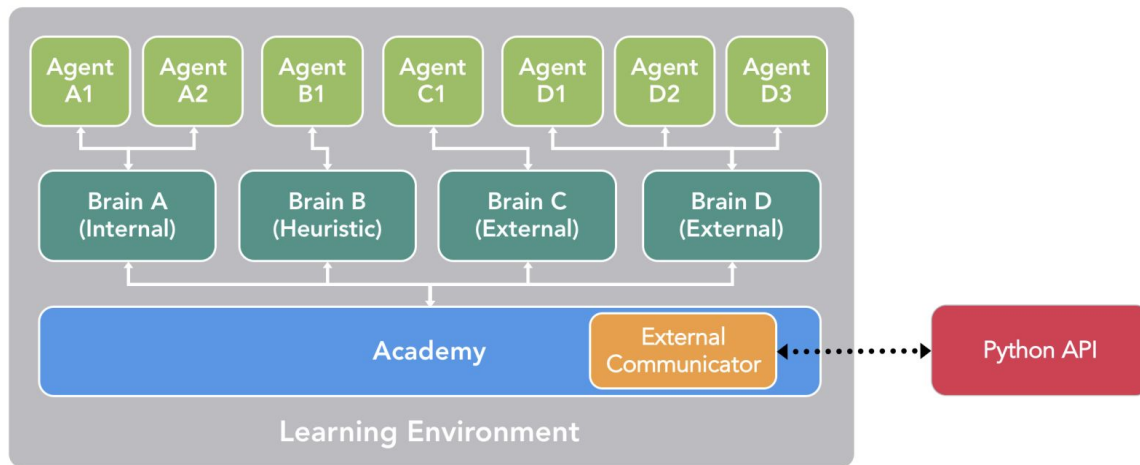






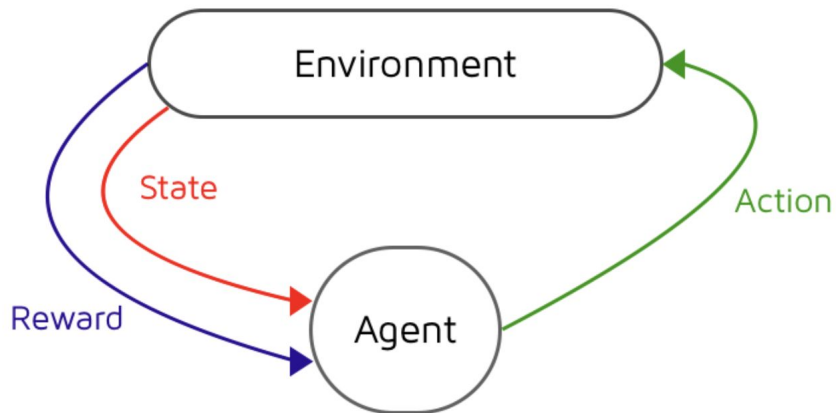
# ML-Agents Toolkit

- Bibliothek für Machine Learning in Unity
- Fokus auf Reinforcement Learning
- Beta-Phase, aber bereits gut dokumentiert

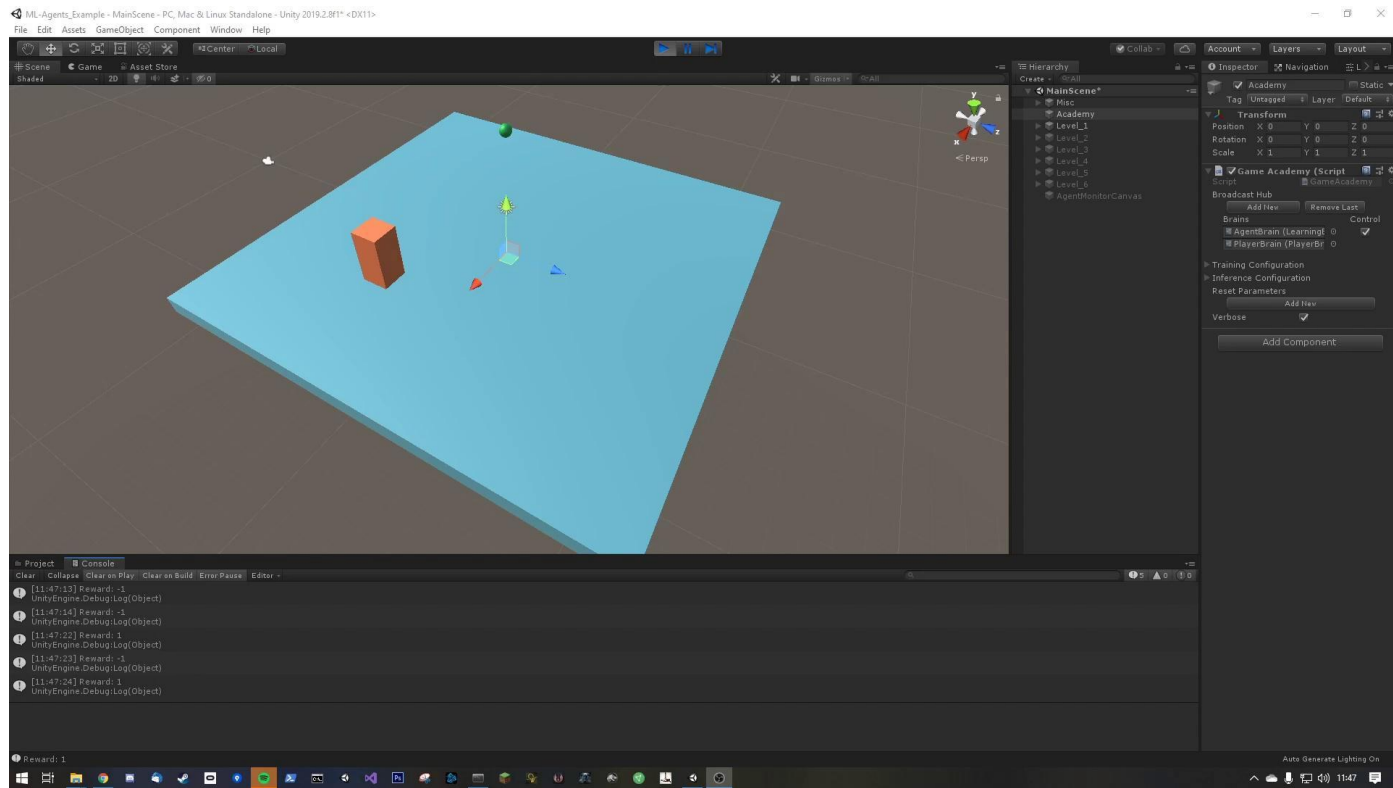


# ML-Agents Toolkit

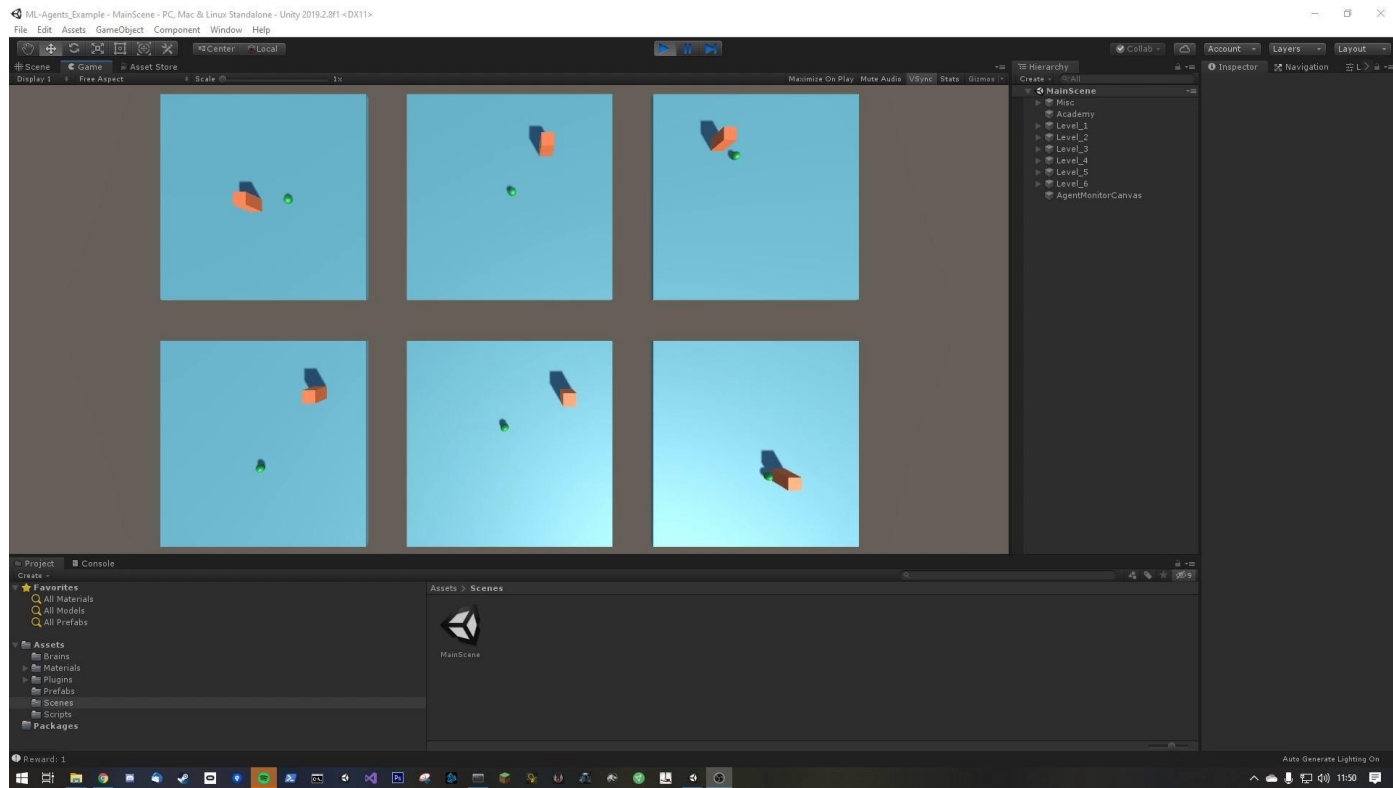
- Überschreiben von Funktionen der Klasse *Agent*
  - *InitializeAgent*
  - *CollectObservations*
  - *AgentAction*
  - *AgentReset*



# ML-Agents Toolkit - Beispiel Training

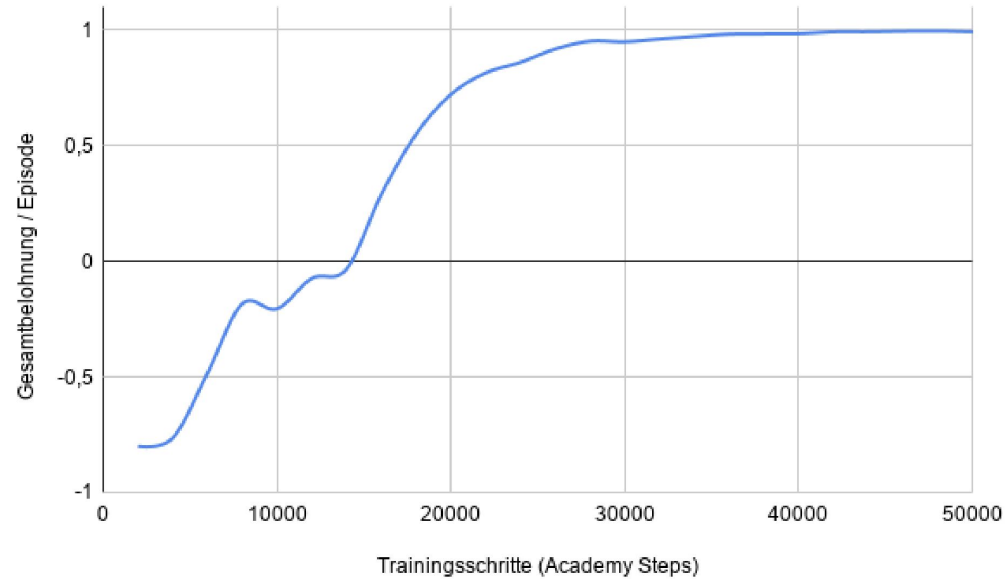


# ML-Agents Toolkit-Beispiel





# ML-Agents Toolkit



# 4. Dorfsystem

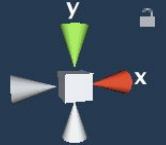


# Modellierung des Dorfes

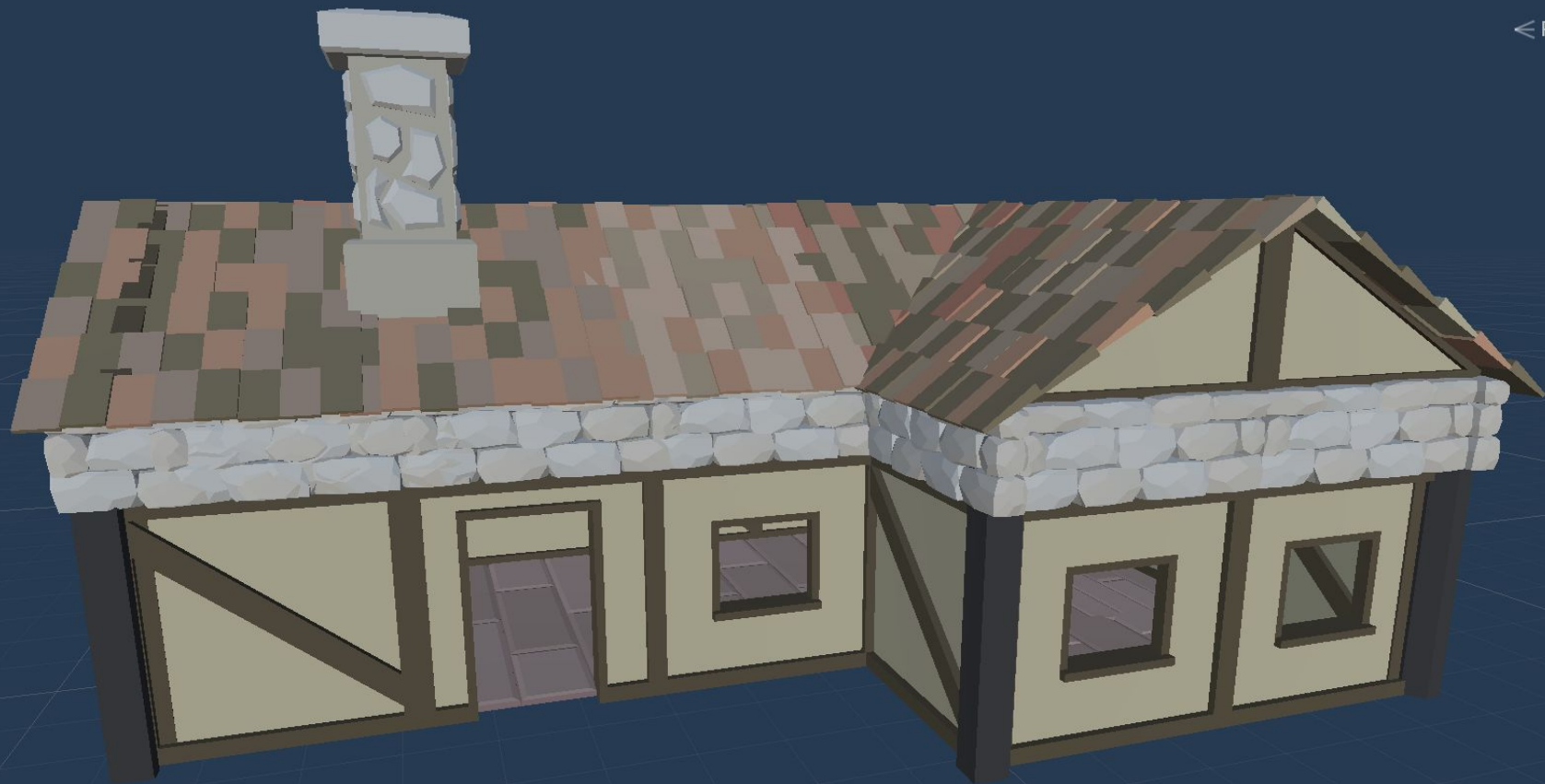
- Dorfumgebung musste geschaffen werden
- Kostenlose geeignete Gebäudemodelle waren nicht verfügbar
- Modellierung eigener Gebäude aus modularen Komponenten



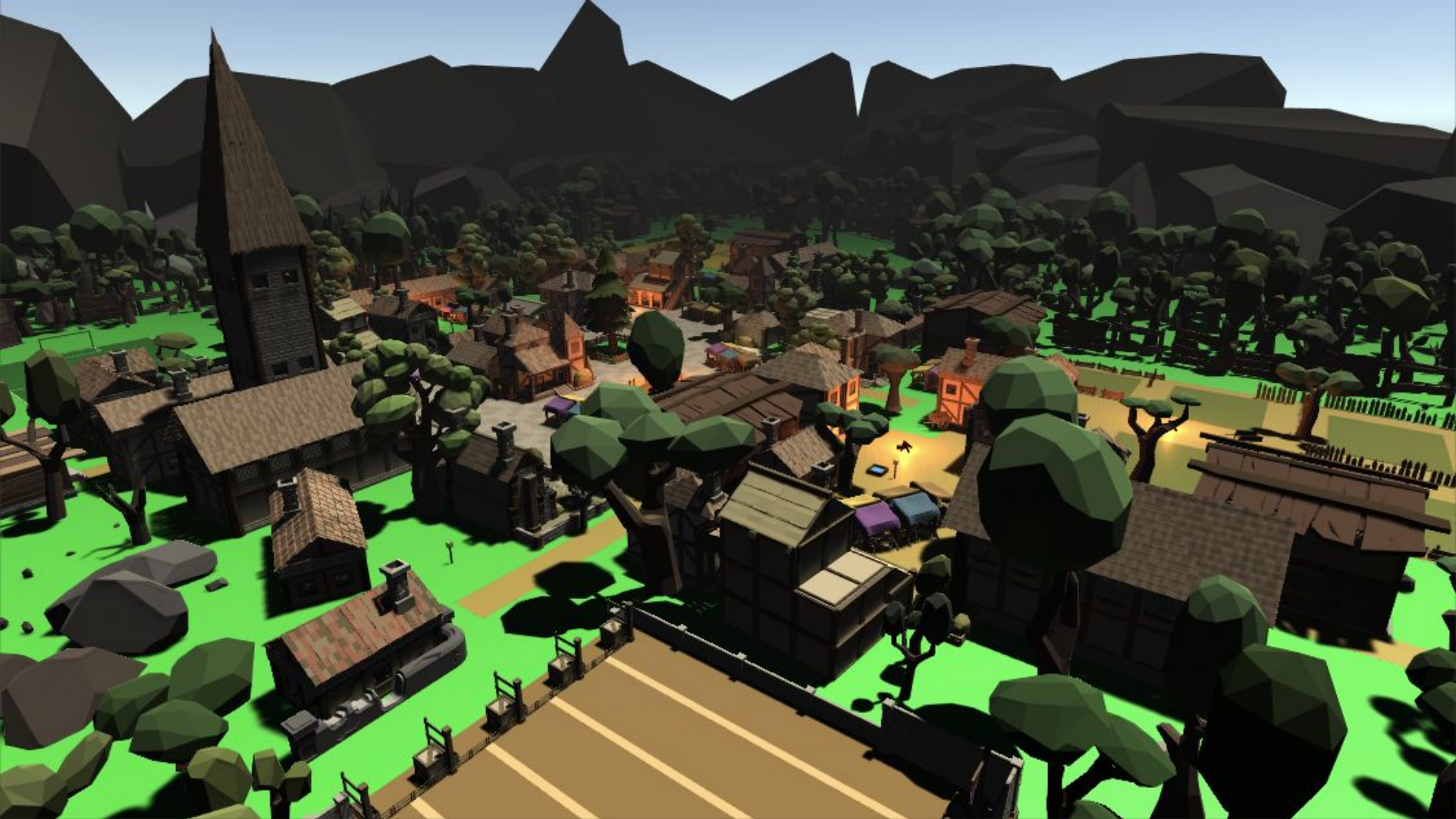


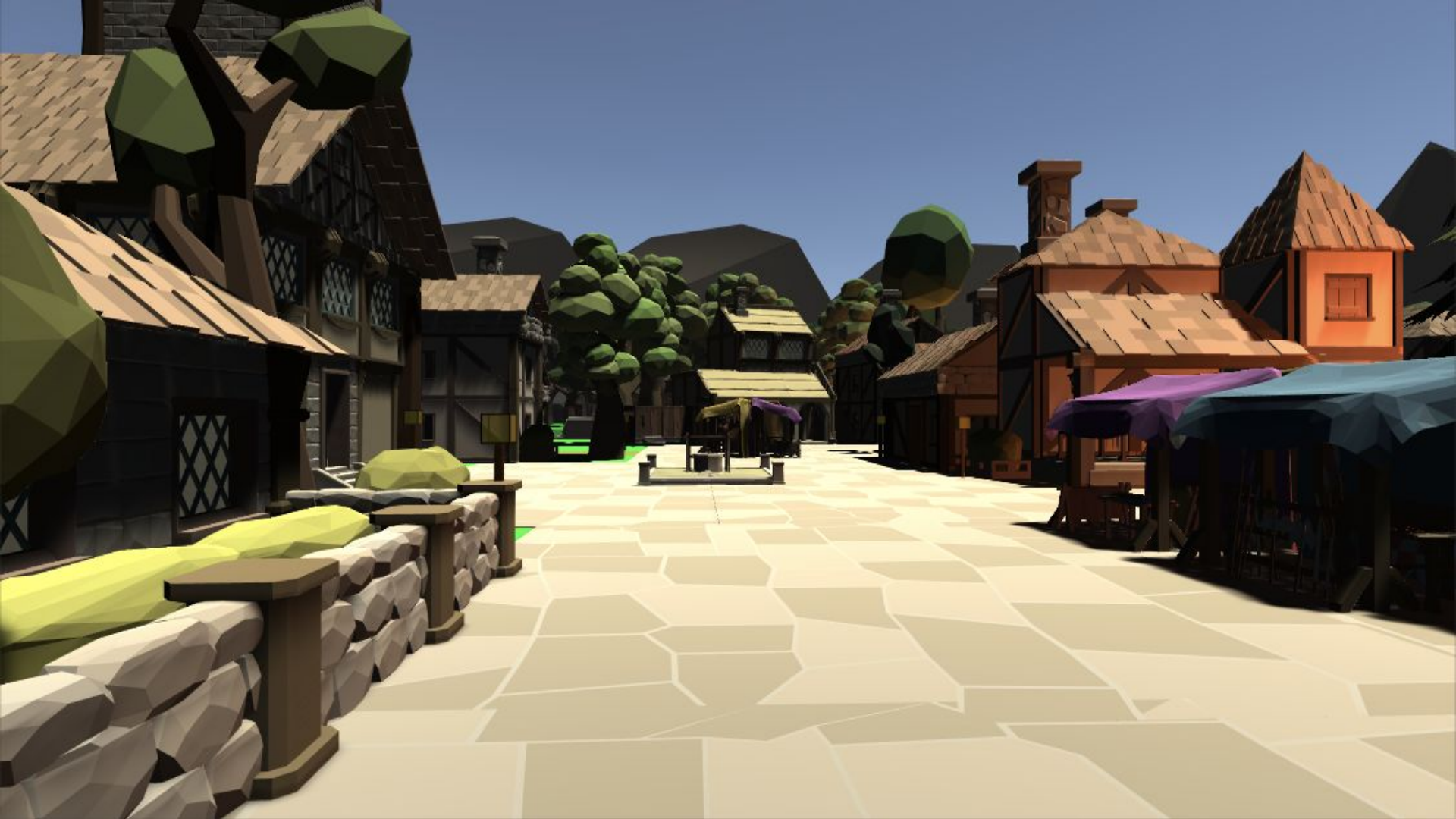


← Persp









# Funktionsweise

- Zeit schreitet voran (durch *TimeController*)
  - Visueller Tagesablauf durch Lichtquellen und Sonne/Himmel
  - *NpcState* wird verändert, Bedürfnisse nehmen mit der Zeit ab





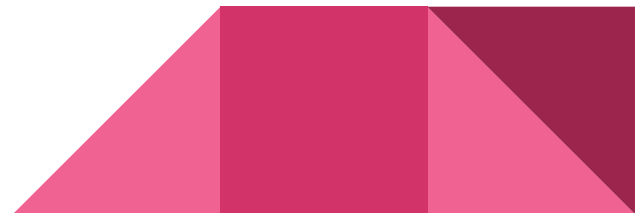






# Funktionsweise

- Zeit schreitet voran (durch *TimeController*)
  - Visueller Tagesablauf durch Lichtquellen und Sonne/Himmel
  - *NpcState* wird verändert, Bedürfnisse nehmen mit der Zeit ab
- Agenten führen kontinuierlich Aktionen aus, um sinkende Bedürfnisse zu befriedigen



# Funktionsweise

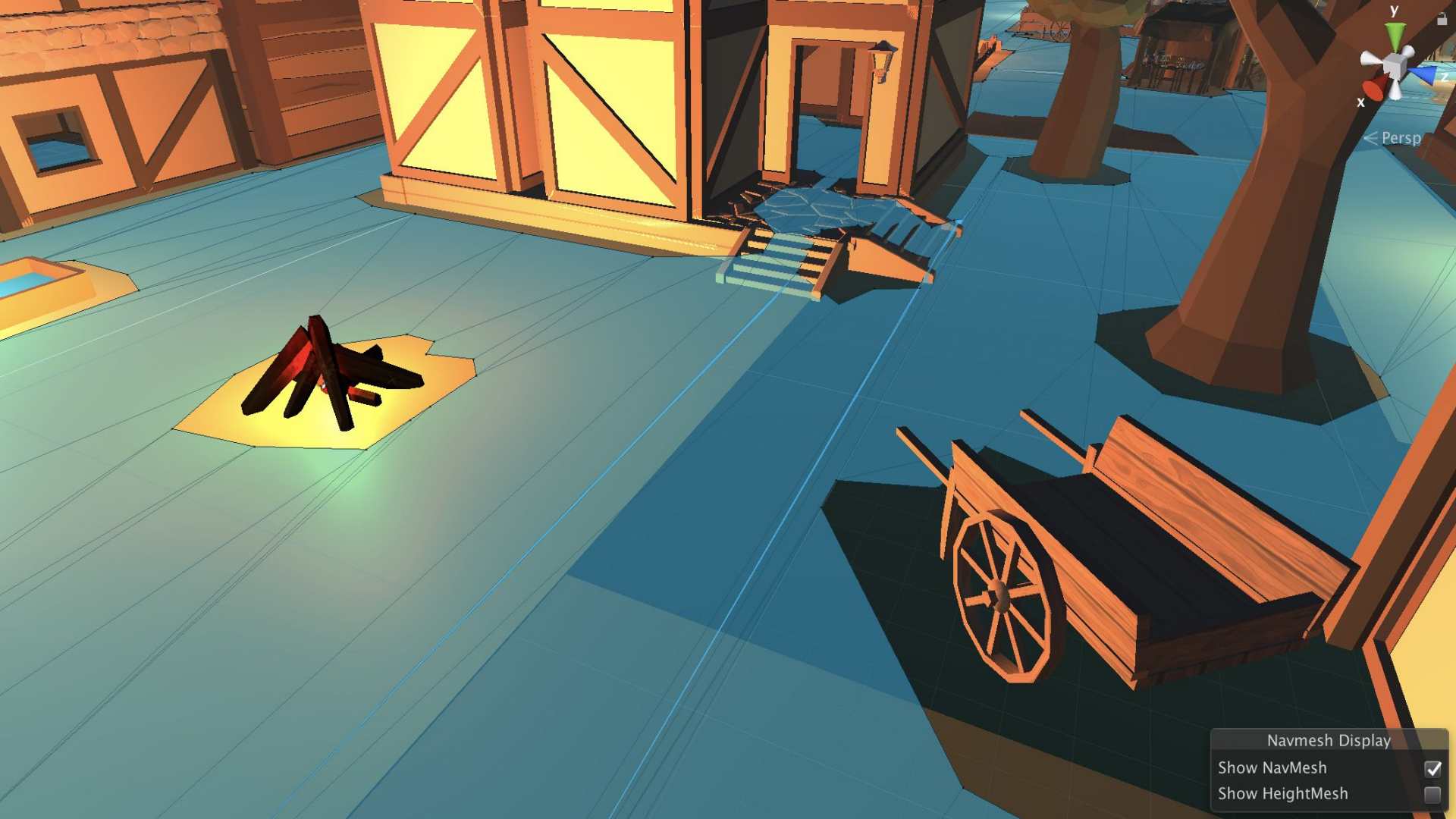
- Zeit schreitet voran (durch *TimeController*)
  - Visueller Tagesablauf durch Lichtquellen und Sonne/Himmel
  - *NpcState* wird verändert, Bedürfnisse nehmen mit der Zeit ab
- Agenten führen kontinuierlich Aktionen aus, um sinkende Bedürfnisse zu befriedigen
- Nicht ausführbare Aktionen werden maskiert



# Funktionsweise

- Zeit schreitet voran (durch *TimeController*)
  - Visueller Tagesablauf durch Lichtquellen und Sonne/Himmel
  - *NpcState* wird verändert, Bedürfnisse nehmen mit der Zeit ab
- Agenten führen kontinuierlich Aktionen aus, um sinkende Bedürfnisse zu befriedigen
- Nicht ausführbare Aktionen werden maskiert
- Aktionen beinhalten Zielorte, Navigation erfolgt über Unity's *NavMesh*





< Persp

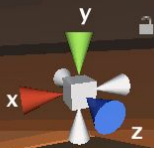
Navmesh Display

Show NavMesh



Show HeightMesh





Navmesh Display

|                 |                                     |
|-----------------|-------------------------------------|
| Show NavMesh    | <input checked="" type="checkbox"/> |
| Show HeightMesh | <input type="checkbox"/>            |

# ActionPlans

- Eigene Daten-/Kontrollstruktur für Ausführung von Aktionen: *ActionPlan*
- Jede Aktion verfügt über einen ActionPlan
- ActionPlan enthält sequentielle Subaktionen mit Zielorten
- Callback nach jeder Subaktion, um nächste Aktion zu starten
- Berechnung von Belohnung und Bestrafung am Ende jeder Aktion

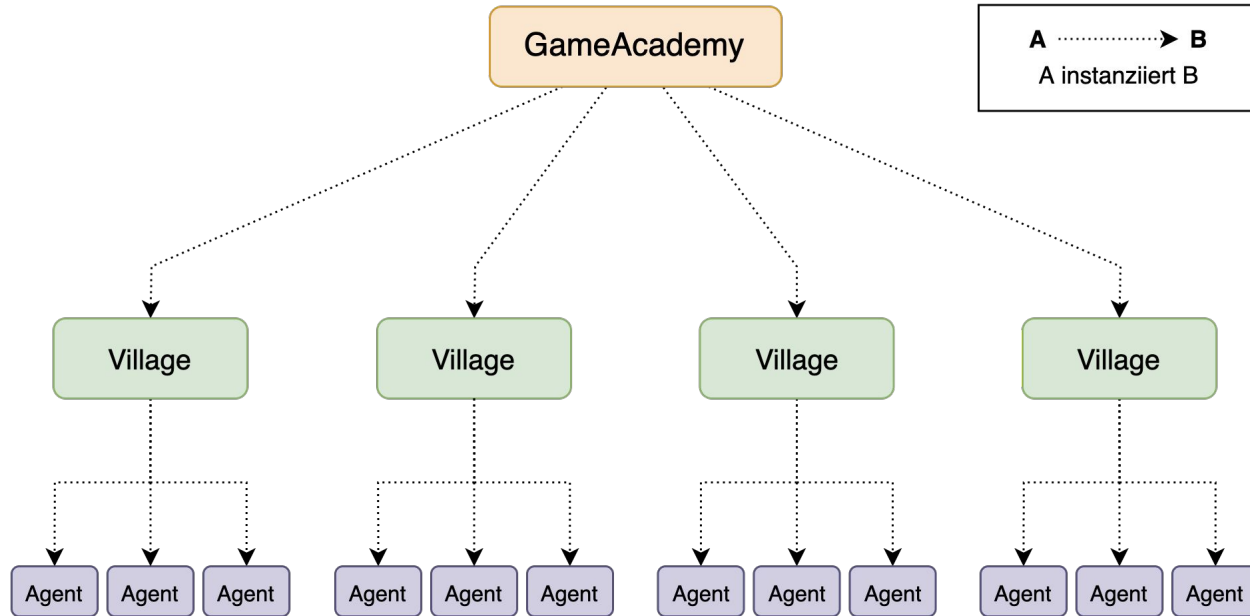


# Agenten

- Führen kontinuierlich Aktionen aus
- Bedürfnisse
  - *Hunger, Durst, Schlaf, Arbeit, Bildung, Kommunikation, Glauben*
- Attribute
  - *Fitness, Intelligenz, Gemeinschaftsgefühl, Konsum, Wildheit*
- Agenten haben Berufe, beeinflussen leicht Attribute
- Attribute und Bedürfnisse wirken sich auf Belohnungen aus
- Wenn Bedürfnisse befriedigt sind,  
Werden Freizeitaktionen ausgeführt



# Architektur

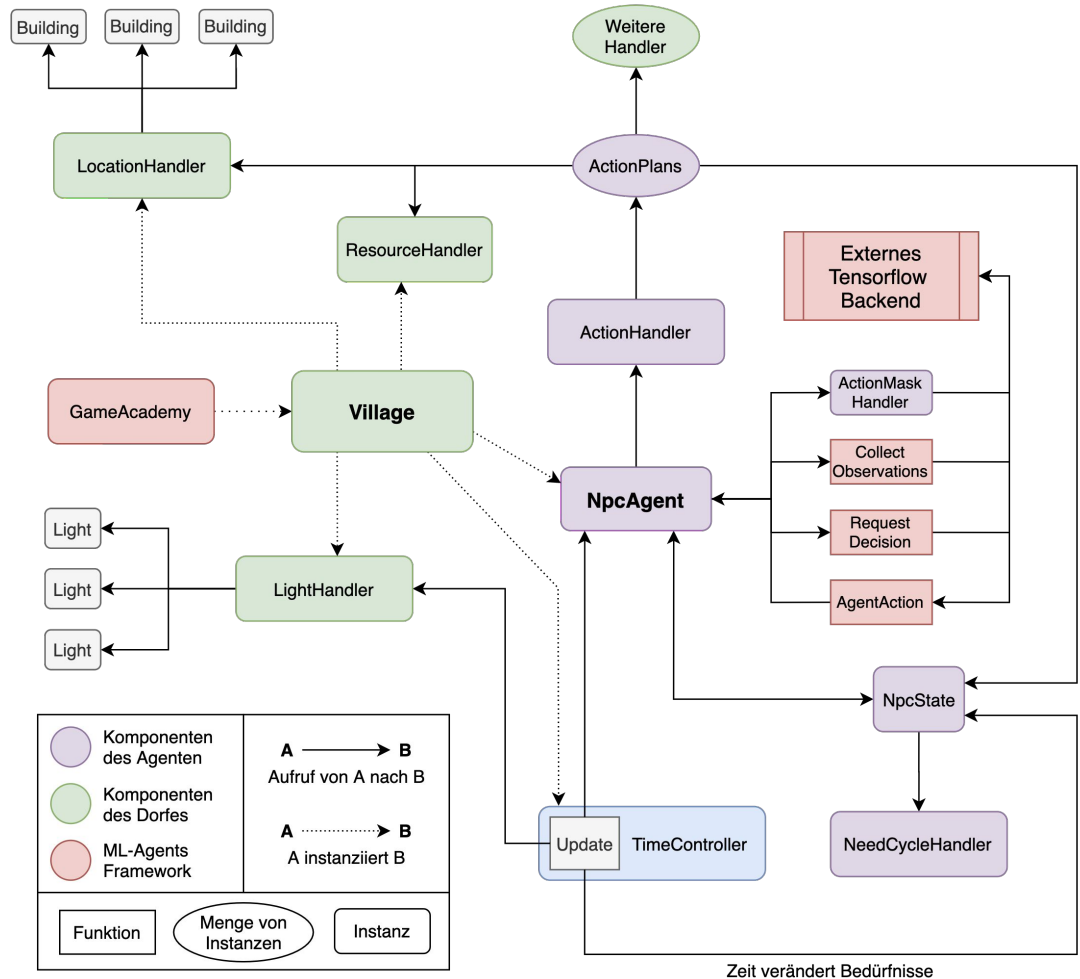




# Architektur

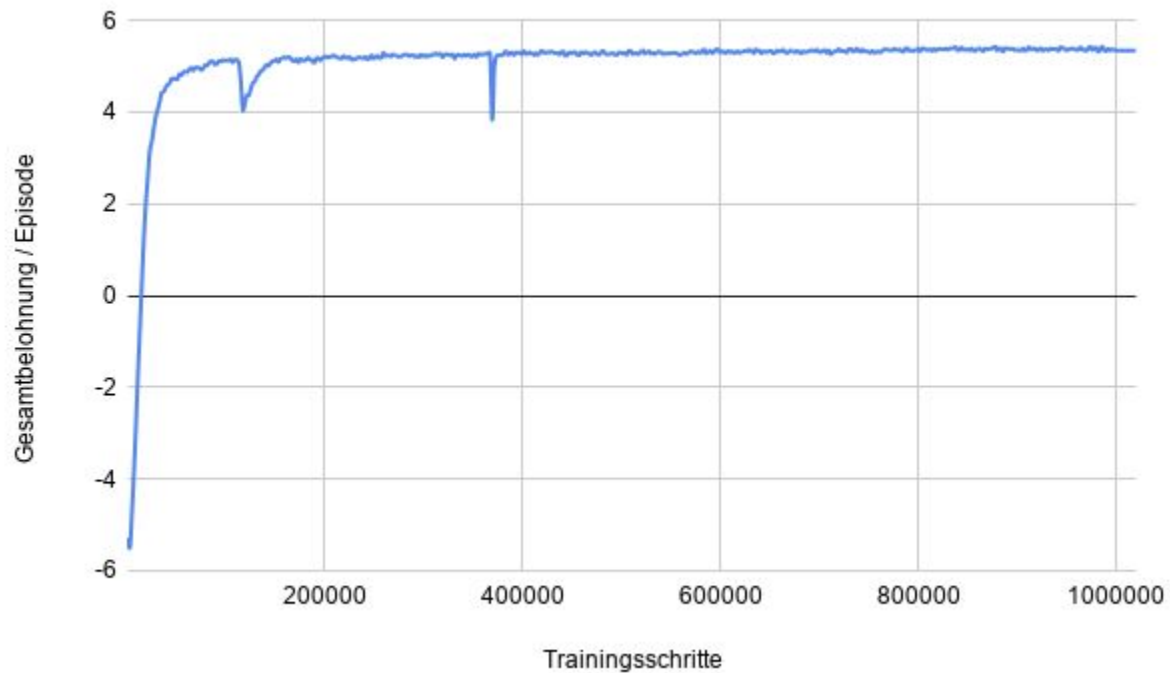
## - Zentrale Komponenten

- GameAcademy
- Village
- NpcAgent
- Tensorflow Backend
- TimeController
- ActionPlans

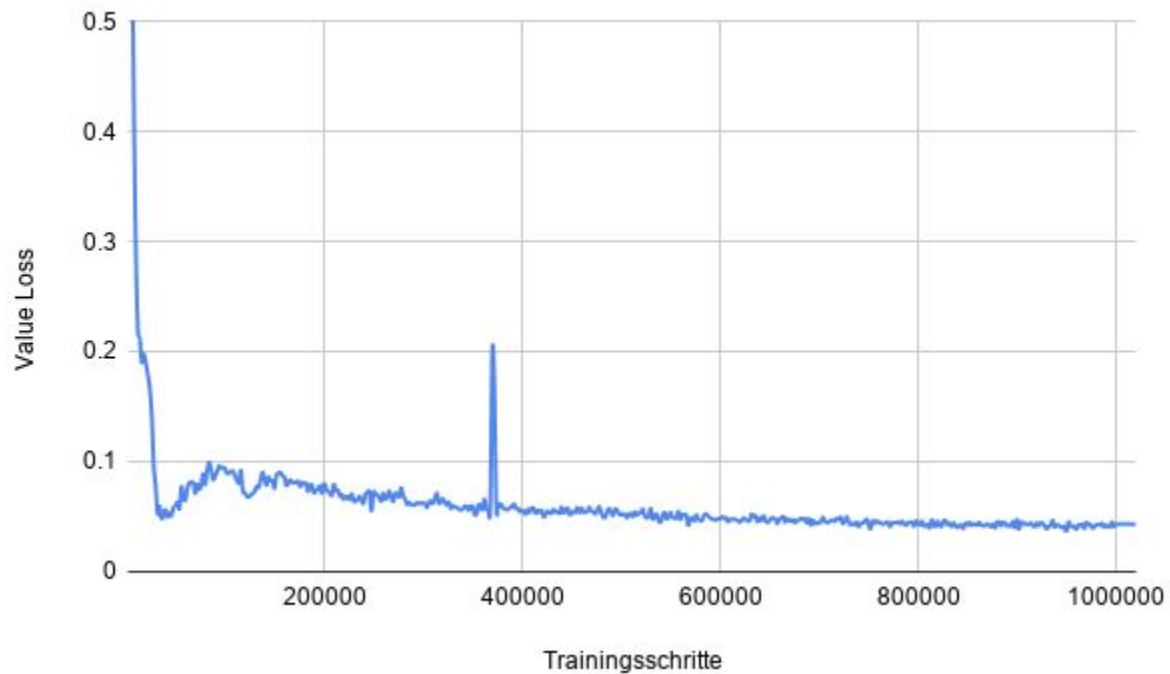


# 5. Ergebnisse

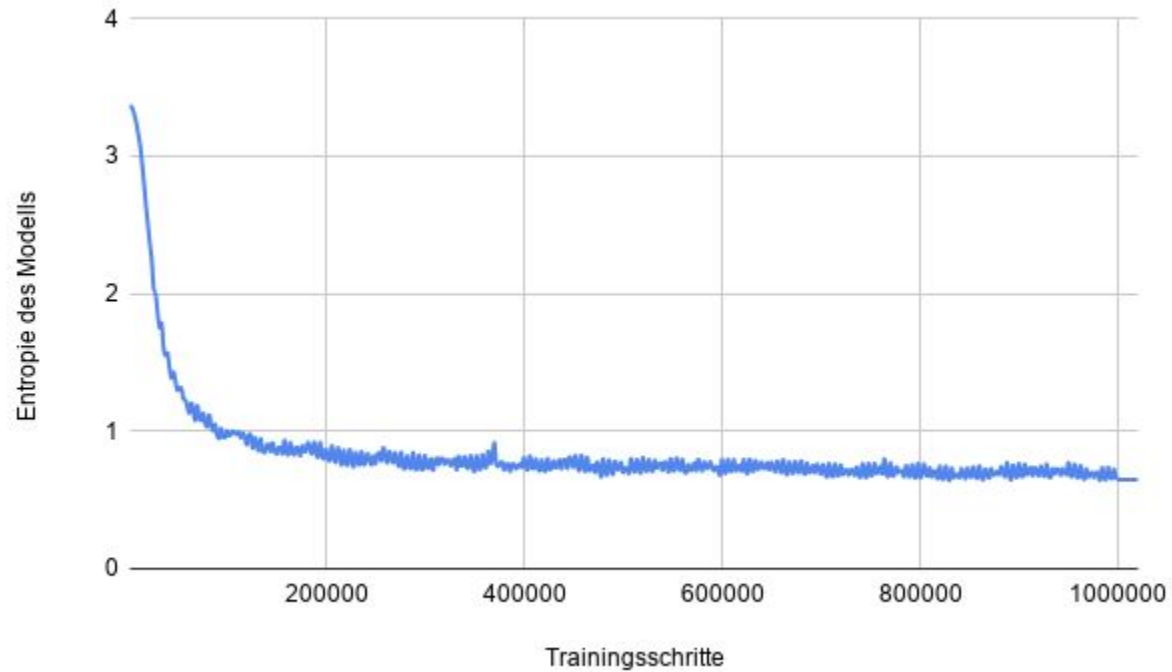
# Belohnungsverlauf



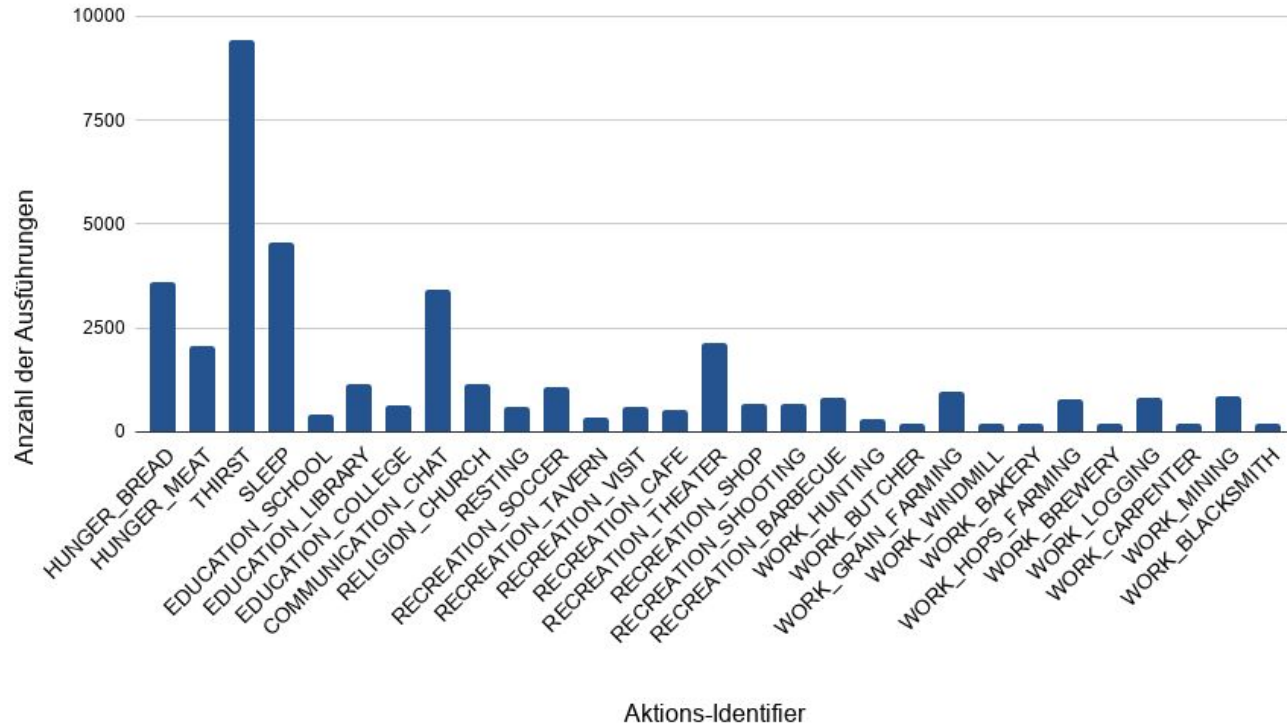
# Value Loss



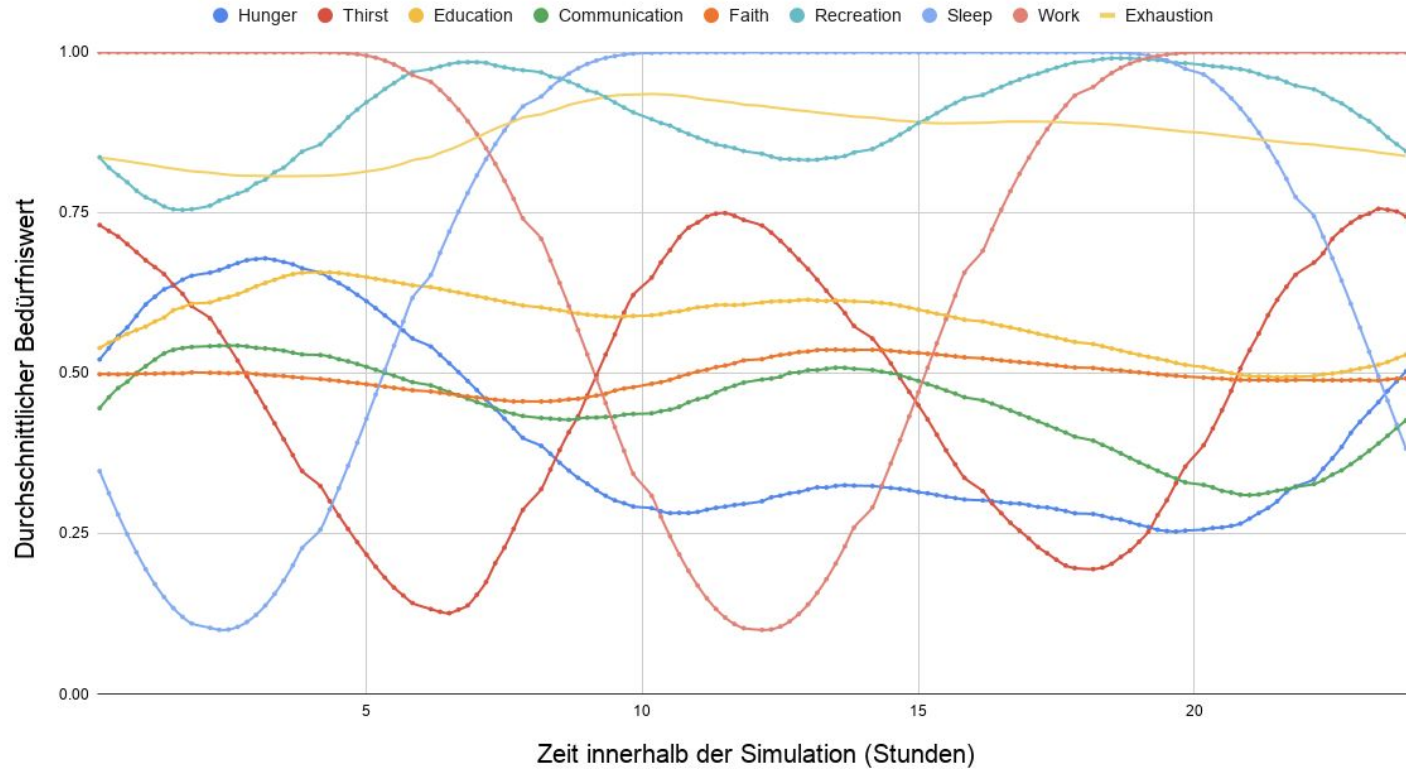
# Entropieverlauf



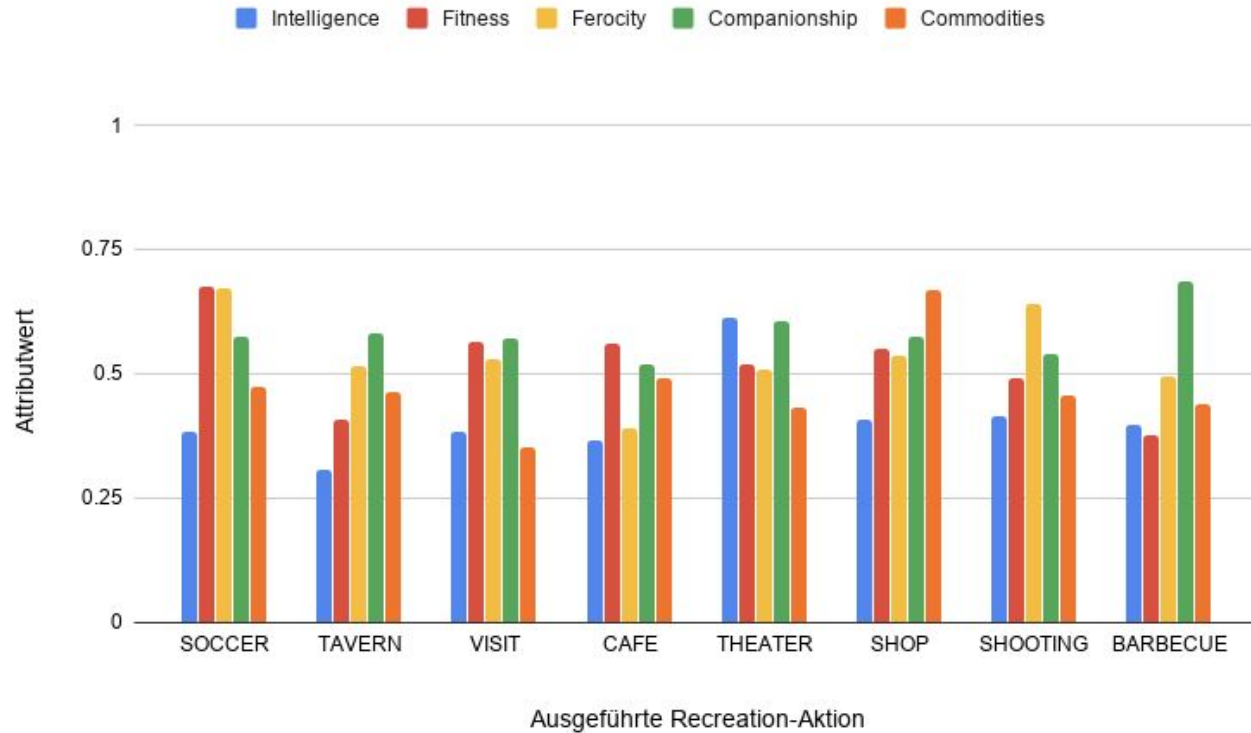
# Verhältnis ausgeführter Aktionen



# Bedürfnisverlauf



# Verhältnis ausgeführter Aktionen



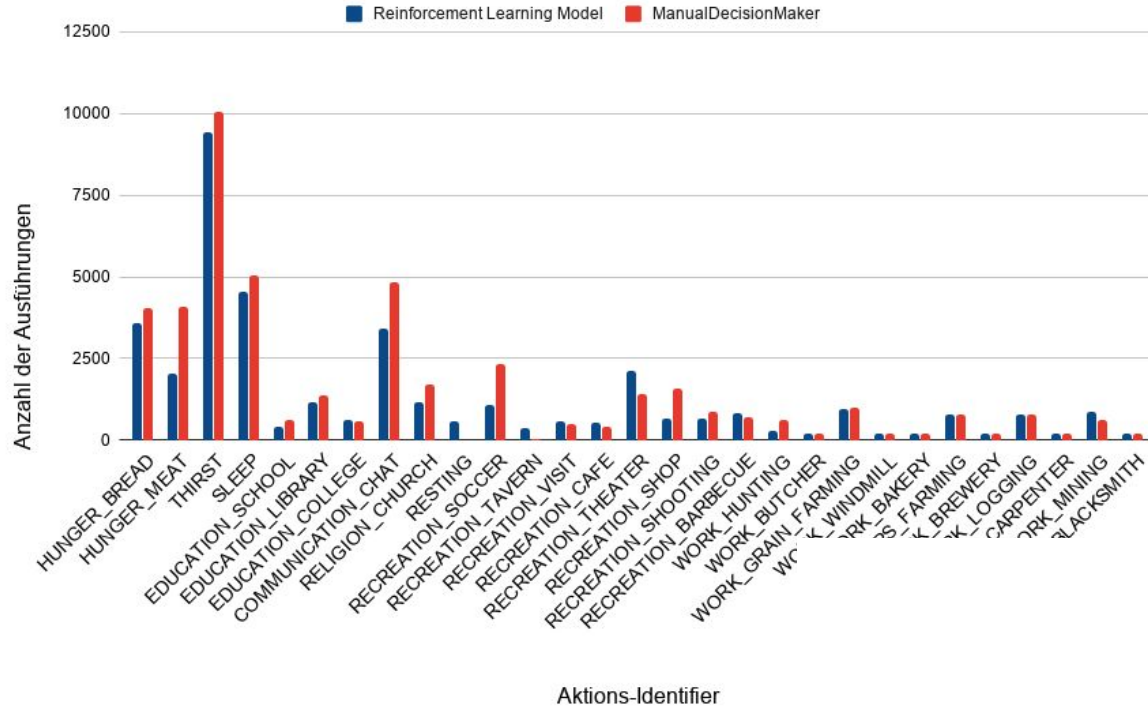


# Alternative Implementierung

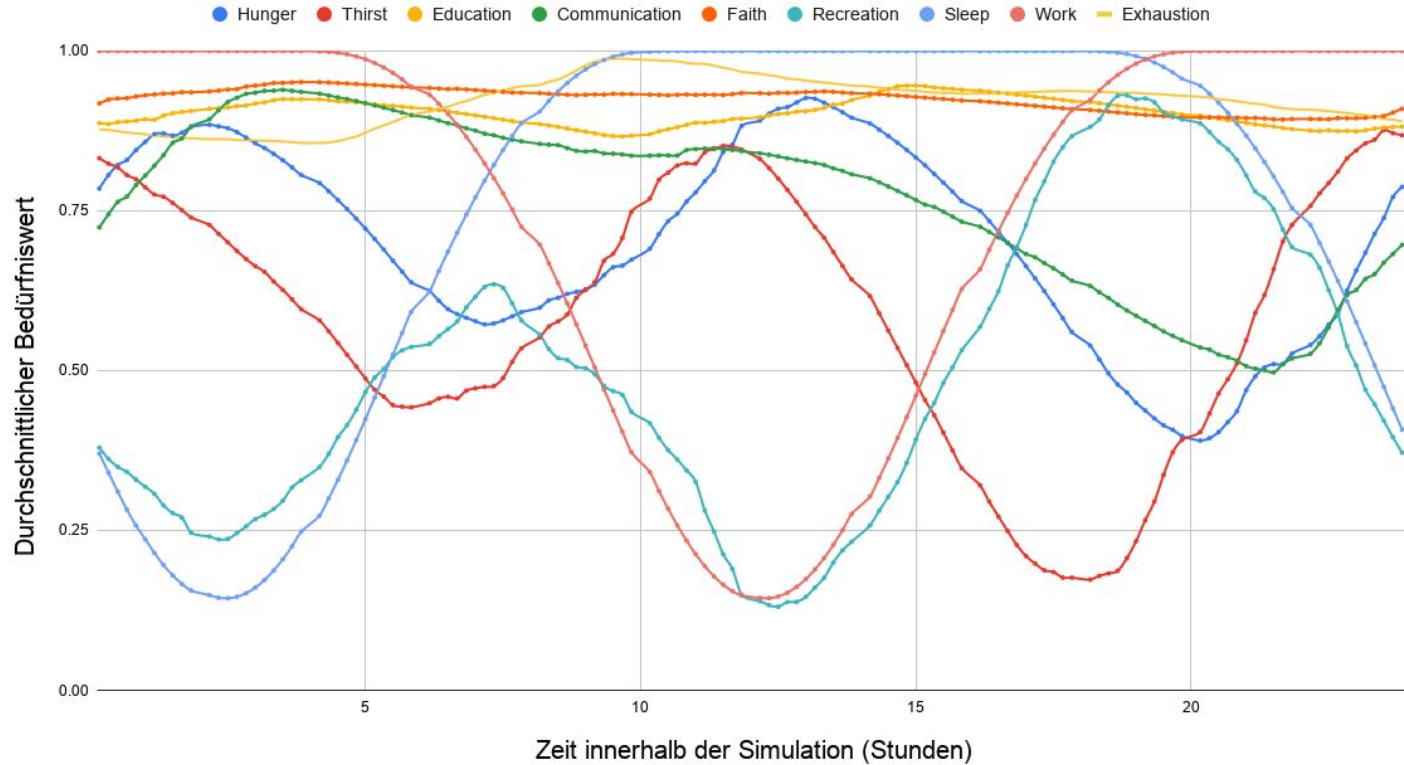
- Regelbasierter Ansatz wurde implementiert
- Erzielt ähnliche Ergebnisse
- Problem: Regeln können sich widersprechen, schlechter erweiterbar



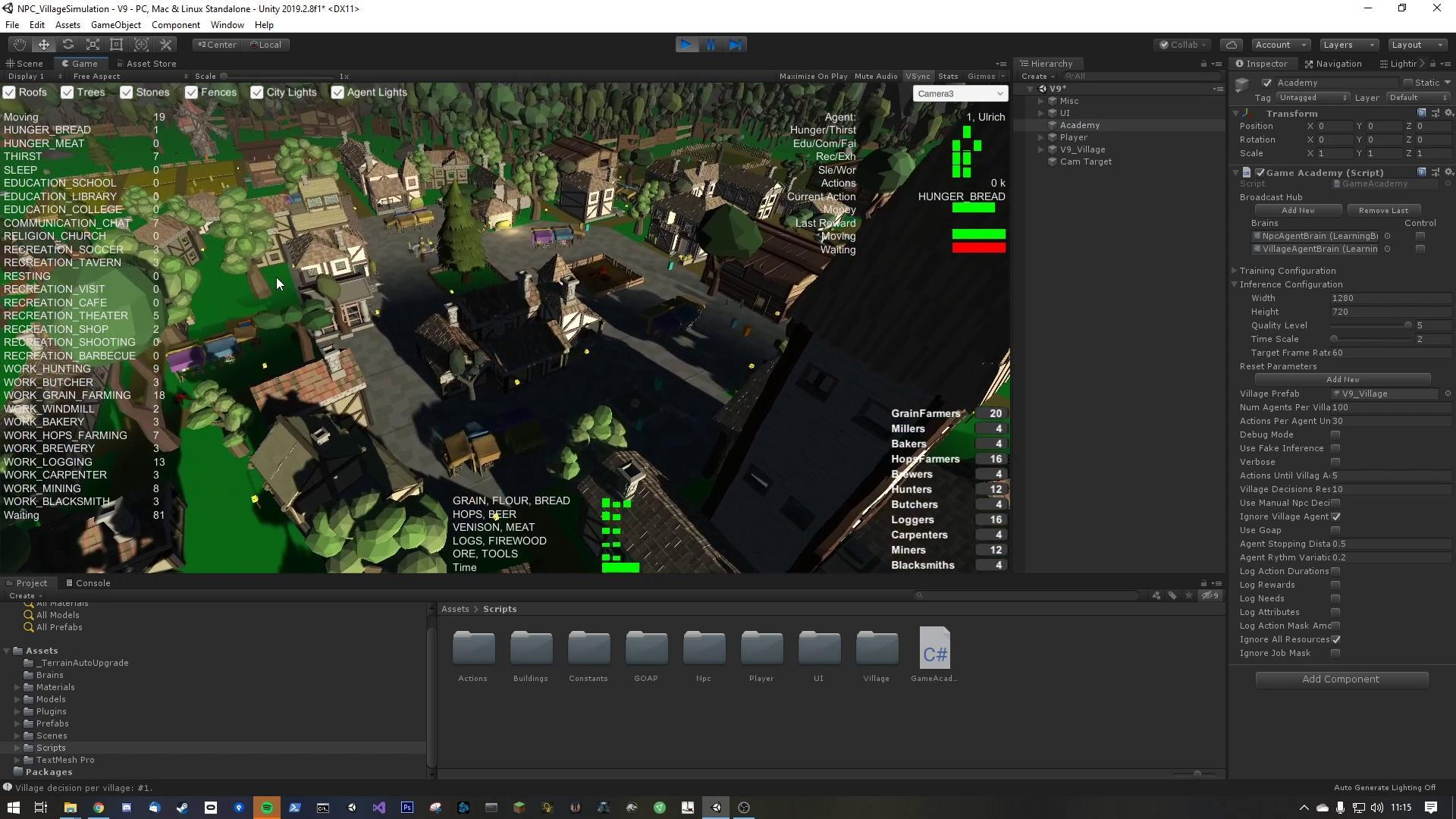
# Alternative Implementierung



# Alternative Implementierung



# 6. Demonstration



# 7. Fazit

# Fazit - Zielsetzung

- Implementierung eines funktionierenden Dorfsystems ✓
  - Dorfsystem mit intelligenten Agenten wurde entwickelt
- Dorf wird von NPCs mit Bedürfnissen bevölkert ✓
  - Dorf wirkt lebendig und ist mit NPCs gefüllt
- Verhalten der NPC soll authentischer und weniger statisch wirken ✓
  - NPCs haben gelernt, sinnvolle Entscheidungen aufgrund ihrer Bedürfnisse zu treffen
  - Verhalten ist weniger statisch, da nicht von Hand gescrripted



# Fazit - Zielsetzung

- Entstehung eines Tagesablaufs
  - Agenten haben groben Tagesablauf entwickelt ✓
  - Schlaf- und Arbeitsbedürfnis mussten vorgegeben werden ✓
- Reduzierung des Aufwands
  - Kein Scripting nötig, dafür viel Trainingsaufwand ✓
  - Agenten sind gut erweiterbar ✓





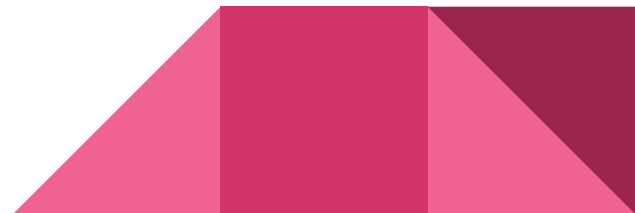
# Fazit - Zielsetzung

- Entstehung eines Tagesablaufs
  - Agenten haben groben Tagesablauf entwickelt ✓
  - Schlaf- und Arbeitsbedürfnis mussten vorgegeben werden ✓
- Reduzierung des Aufwands
  - Kein Scripting nötig, dafür viel Trainingsaufwand ✓
  - Agenten sind gut erweiterbar ✓



# Probleme

- Nichtdeterminismus als Hindernis
- Probleme mit ML-Agents (Beta)
- Dauer des Trainings



# Ausblick

- Gemeinschaftsgefühl
- Funktionierendes Ressourcensystem
- Mehrere Dörfer, die miteinander interagieren (z.B. Handel)
- Ökosystem
- Reaktion auf äußere Einflüsse
- Mehr Freiheiten für Agenten





# Fragen?